ZIP

yelpcamp_analysis

property_sharing_and_visitor_stories_p...

TXT

To build a production-grade Retriev

A lot feature are implement and yet to be implemented , implemented features are broken {either does not work pproperty like chat system a lot to imporve as given in upload files} , critque all of previous code , also see To be added feature in both plans / upload context , give detailed task step wiht exceptional specificaitn both for frontend and backend separte yet keeping both compatible

The uploaded archive (`source: 1`) suffered binary zip corruption during upload, leaving only the file tree headers intact (`backend/*` and `frontend-ver2/*`) while stripping the raw file contents. However, inspecting that exact file manifest against your two architectural specifications (`property_sharing_and_visitor_stories_plan.md` and `To build a production-grade Retriev.txt`) reveals every structural disconnect, broken pipeline, and missing module in your current build. ZIP + 2

Part 1: Critique of Current Codebase & Root Cause Analysis

1. Why the Chat System & RAG Pipeline Are Broken

Split-Brain Database Engines (`database.py` vs. `turso_engine.py`): Your backend separates standard SQLModel ORM operations (`database.py`) from raw libSQL vector queries (`turso_engine.py`). Because SQLite/Turso locks or desynchronizes when two different clients write and read across unaligned schemas, newly created campgrounds, comments, and stories in `models.py` are not automatically populating or updating the `F32_BLOB(768)` column required by `vector_top_k` . ZIP + 2

Pure Bi-Encoder Vector Trap (`vector_store.py` & `rag_service.py`): Your current `vector_rag.py` and `rag_service.py` rely on pure cosine similarity (`vector_top_k`). As outlined in your architecture notes, pure vector search fails on

negations (e.g., *"quiet camp with NO loud music"* matches DJ party camps at ~88% similarity), **rare proper nouns** (e.g., *"Munsiyari Khaliya Top"* gets buried under generic *"Manali"* posts because it lacks SQLite FTS5 BM25 keyword weighting), and **unbounded UI matches** (returning the closest button even when cosine distance is `> 0.25`). ZIP + 2

Disconnected Chat UI (`AIAssistantDock.jsx` vs. `vector_rag.py`): `AIAssistantDock.jsx` is built as a passive text-chat widget. You have **not** created `AgentProvider.jsx` or `useAgentAction.js` in `frontend-ver2/src/context/`. Consequently, the chat assistant is completely blind to the active route, open modals, or form states, and cannot execute frontend callbacks or highlight DOM elements via `driver.js`. ZIP + 3

Synchronous Blocking on Media Ingestion: Calling `multimodal_embedder.py` synchronously during post or photo creation blocks the FastAPI event loop, causing request timeouts on the frontend when Gemini Vision + Gemma Embeddings run sequentially. ZIP + 1

2. Missing & Half-Implemented Features Across Plans

Missing Backend Modules: `backend/services/vision_service.py` and `backend/routers/visitor_photos.py` (specified in Plan 1) do not exist in the `backend/` directory. `social.py` currently attempts to shoulder social interactions without dedicated `VisitorPhoto` and `VisitorPhotoLike` table schemas or 24-hour TTL (`expires_at`) filtering. ZIP + 4

Flat Comments Instead of Recursive Adjacency Lists: `backend/routers/comments.py` handles flat reviews rather than an adjacency list (`parent_comment_id`) with a debounced background worker that aggregates 5+ nested comments into a `community_summary` vector. ZIP + 1

Missing Frontend Components: `frontend-ver2/src/components/` has `CamperStories.jsx` (static showcase), but lacks the interactive `StoryBar.jsx`, `StoryViewerModal.jsx` (with 5-second auto-advancing progress bars), and `SharePropertyModal.jsx` (with canvas QR code and Open Graph preview). ZIP + 1

Part 2: Backend Master Specification (`backend/`)

Task B1: Unified Schema & Turso Hybrid Search Initialization

Files to Modify: `backend/models.py`, `backend/turso_engine.py`, `backend/schemas.py`

ZIP

1. **Extend `Campground` Model (`backend/models.py`):**
   Add JSON-serialized/text columns and vector fields so ORM and libSQL share a single source of truth: MD + 1

Python

```python
class Campground(SQLModel, table=True):
    __tablename__ = "campground"
    id: Optional[int] = Field(default=None, primary_key=True)
    title: str = Field(index=True)
    price: float
    description: str
    location: str
    location_address: Optional[str] = Field(default="")
    image: str
    gallery_images: str = Field(default="[]")  # JSON list of URLs
    amenities: str = Field(default="[]")       # JSON list: ["wifi", "campfir
    rules: Optional[str] = Field(default="")   # Check-in rules, max guests,
    avg_rating: float = Field(default=0.0)
    ai_media_caption: Optional[str] = Field(default="")
    community_summary: Optional[str] = Field(default="")
    author_id: int = Field(foreign_key="user.id")
```

2. **Add `VisitorPhoto` & `VisitorPhotoLike` Models (`backend/models.py`):**
   Include `expires_at` (Unix timestamp for 24h stories; `NULL` for permanent highlights) to satisfy both Plan 1 and Plan 2: MD + 1

Python

```python
class VisitorPhoto(SQLModel, table=True):
    __tablename__ = "visitor_photos"
    id: Optional[int] = Field(default=None, primary_key=True)
    campground_id: int = Field(foreign_key="campground.id", index=True)
    user_id: int = Field(foreign_key="user.id", index=True)
    user_username: str
    user_avatar: Optional[str] = Field(default=None)
    media_type: str = Field(default="photo")   # "photo" | "video"
    image_url: str
```

```
    caption: Optional[str] = Field(default="")
    ai_tags: str = Field(default="[]")          # JSON list from Gemini Vision
    ai_media_caption: Optional[str] = Field(default="")
    likes_count: int = Field(default=0)
    created_at: str = Field(default_factory=lambda: datetime.now(timezone.utc
    expires_at: Optional[int] = Field(default=None, index=True) # Unix epoch

class VisitorPhotoLike(SQLModel, table=True):
    __tablename__ = "visitor_photo_likes"
    user_id: int = Field(foreign_key="user.id", primary_key=True)
    photo_id: int = Field(foreign_key="visitor_photos.id", primary_key=True)
```

3. **Update `Comment` Model for Nested Threads (`backend/models.py`):**
   Add
   `parent_comment_id: Optional[int] = Field(default=None, foreign_key="comment.id", index=True)`
   and `rating: Optional[int] = Field(default=None)` . TXT

4. **Provision Vector Columns, DiskANN Indexes, and FTS5 Virtual Tables (`backend/turso_engine.py`):**
   Execute on startup so Turso natively handles both dense vectors and BM25 keyword search : ZIP + 1

SQL

```
ALTER TABLE campground ADD COLUMN embedding F32_BLOB(768);
ALTER TABLE visitor_photos ADD COLUMN embedding F32_BLOB(768);

CREATE INDEX IF NOT EXISTS idx_campground_vec ON campground (libsql_vector_id
CREATE INDEX IF NOT EXISTS idx_stories_vec ON visitor_photos (libsql_vector_i

CREATE VIRTUAL TABLE IF NOT EXISTS campground_fts USING fts5(
    title, description, location, location_address, amenities, ai_media_capti
    content='campground', content_rowid='id'
);

CREATE TABLE IF NOT EXISTS ui_navigation (
    id TEXT PRIMARY KEY,
    feature_name TEXT NOT NULL,
    route TEXT NOT NULL,
    dom_selector TEXT NOT NULL,
    description TEXT NOT NULL,
    embedding F32_BLOB(768)
```

```
);
CREATE INDEX IF NOT EXISTS idx_ui_nav_vec ON ui_navigation (libsql_vector_idx
```

Task B2: Multimodal Vision & Background Ingestion Pipeline

Files to Create/Modify: Create `backend/services/vision_service.py` ; Modify `backend/services/multimodal_embedder.py` & `backend/requirements.txt`

ZIP + 1

1. **Dependencies (`backend/requirements.txt`):** Ensure `google-genai` , `flashrank` , `cloudinary` , and `libsql-experimental` are pinned. ZIP + 2
2. **Build `backend/services/vision_service.py` :**

   Implement `analyze_media_with_gemini(image_bytes: bytes, mime_type: str) -> dict` using `google.genai.Client()` . MD + 1

   Prompt the Gemini Flash model with `response_mime_type="application/json"` to return:

   JSON

   ```
   {
     "suggested_caption": "Atmospheric 1-sentence caption",
     "ai_tags": ["stargazing", "lakefront", "campfire", "pine_forest"],
     "detailed_visual_description": "Terrain, weather, gear, and vibe descr
   }
   ```

3. **Async Background Worker (`backend/services/multimodal_embedder.py`):**

   Implement `async def index_campground_or_story(entity_type: str, entity_id: int, image_url: str, text_context: str)` triggered via FastAPI `BackgroundTasks` . TXT

   Combines `text_context` + Gemini Vision `detailed_visual_description` , generates the 768-d vector via Gemma Embeddings 2, and runs:

```sql
UPDATE campground SET ai_media_caption = ?, embedding = vector32(?) WHER
```

followed by updating `campground_fts` . TXT

Task B3: 3-Stage Hybrid RAG & Dynamic Agent Router

Files to Modify: `backend/services/rag_service.py` , `backend/services/vector_store.py` , `backend/routers/vector_rag.py`

ZIP

1. **Implement 3-Stage Retrieval in** `backend/services/vector_store.py` **:**
   Replace raw `vector_top_k` with the 3-stage pipeline: ZIP + 1

   Stage 1 (Dense Vector Search with Distance Cutoff):
   Query `vector_top_k('idx_campground_vec', vector32(?), 15)` joined with `campground` where `vector_distance_cos(c.embedding, vector32(?)) < 0.35` and `c.avg_rating >= min_rating` . For stories, enforce `(s.expires_at IS NULL OR s.expires_at > unixepoch())` . TXT + 1

   Stage 2 (SQLite FTS5 BM25 Keyword Search):
   Query `SELECT rowid, bm25(campground_fts) as score FROM campground_fts WHERE campground_fts MATCH ? ORDER BY score LIMIT 15` to catch rare proper nouns like *"Munsiyari"* or *"Khaliya Top"*. TXT

   Stage 3 (FlashRank Cross-Encoder Reranking):
   Deduplicate Stage 1 + Stage 2 candidates by `id` , pass `[query, candidate_text]` pairs into `flashrank.Ranker(model_name="ms-marco-MiniLM-L-12-v2").rerank()` , filter out candidates with FlashRank score `< 0.25` (eliminating negation traps like *"no loud music"*), and return the top 5. TXT

2. **Implement UI Navigation Lookup with Strict Cutoff (** `find_ui_action` **):**

```sql
```

```
SELECT u.feature_name, u.route, u.dom_selector, u.description,
       vector_distance_cos(u.embedding, vector32(?)) AS dist
FROM vector_top_k('idx_ui_nav_vec', vector32(?), 1) v
JOIN ui_navigation u ON u.rowid = v.id
WHERE vector_distance_cos(u.embedding, vector32(?)) < 0.25;
```

If `row` is `None`, return `{"action": None}` so the assistant never highlights the wrong button. TXT

3. **Unify the Chat Endpoint (`POST /api/agent/interact` in `backend/routers/vector_rag.py`):**
   Accept both the user query and the live React UI snapshot: ZIP + 1

Python

```python
class AgentInteractRequest(BaseModel):
    user_message: str
    ui_context: Dict[str, Any] = {}
    available_actions: List[Dict[str, Any]] = []
    chat_history: List[Dict[str, str]] = []

class AgentInteractResponse(BaseModel):
    reply: str
    campsite_results: List[Dict[str, Any]] = []
    trigger_action_id: Optional[str] = None
    action_args: Optional[Dict[str, Any]] = None
    ui_highlight: Optional[Dict[str, str]] = None # {route, dom_selector, fea
```

Inside the handler, provide Gemini with both backend tools (`search_campsites_and_stories`, `find_ui_action`) and the dynamic `available_actions` passed from `AgentProvider`, returning the strictly typed `AgentInteractResponse`. TXT

Task B4: Visitor Stories, Rich Hosting & Recursive Comments Endpoints

Files to Create/Modify: Create `backend/routers/visitor_photos.py`; Modify `backend/routers/campgrounds.py`, `backend/routers/comments.py`, `backend/main.py`

ZIP + 1

Endpoint	Method	Router File
/api/campgrounds/{id}/photos	POST	visitor_photos.py
/api/campgrounds/{id}/stories	GET	visitor_photos.py
/api/stories/explore	GET	visitor_photos.py
/api/photos/{photo_id}/like	POST	visitor_photos.py

Endpoint	Method	Router File
`/api/photos/{photo_id}`	`DELETE`	`visitor_photos.py`
`/api/campgrounds` & `/{id}`	`POST` / `PUT`	`campgrounds.py`
`/api/campgrounds/{id}/comments`	`POST`	`comments.py`

Register `visitor_photos.router` in `backend/main.py` . ZIP + 1

Part 3: Frontend Master Specification (`frontend-ver2/`)

Task F1: Dynamic Agent State Machine & Driver.js Integration

Files to Create/Modify:

Create `frontend-ver2/src/context/AgentContext.jsx`

Create `frontend-ver2/src/hooks/useAgentAction.js`

Create `frontend-ver2/src/hooks/useAssistantActions.js`

Modify `frontend-ver2/src/App.jsx` and `frontend-ver2/src/components/AIAssistantDock.jsx` ZIP + 1

1. **Install** `driver.js` **(** `package.json` **):** Add `"driver.js": "^1.3.1"` . ZIP + 1
2. **Build** `AgentContext.jsx` **(** `frontend-ver2/src/context/AgentContext.jsx` **):**

   Maintain `registeredActions` (`Record<string, { id, description, parameters, handler }>`) and `uiContext` (`current_route` , `active_campground_id` , `active_campground_title` , `open_modal` , `draft_fields`). TXT

   Automatically sync `current_route` via `useLocation()` from `react-router-dom` .

   Expose `registerAction(action)` , `updateUiContext(key, val)` , `executeAction(actionId, args)` , and `getCurrentSnapshot()` . TXT
3. **Build** `useAssistantActions.js` **(** `frontend-ver2/src/hooks/useAssistantActions.js` **):**

   Handle `ui_highlight` payloads returned from `/api/agent/interact` : TXT

JavaScript

```
import { useNavigate } from 'react-router-dom';
import { driver } from 'driver.js';
import 'driver.js/dist/driver.css';

export function useAssistantActions() {
  const navigate = useNavigate();
  const executeUiHighlight = (highlight) => {
    if (!highlight?.dom_selector) return;
    if (highlight.route && window.location.pathname !== highlight.route)
      navigate(highlight.route);
    }
    setTimeout(() => {
```

```
      const el = document.querySelector(highlight.dom_selector);
      if (!el) return;
      const drv = driver({ animate: true, showProgress: false });
      drv.highlight({
        element: highlight.dom_selector,
        popover: { title: highlight.feature_name, description: highlight
      });
    }, 350);
  };
  return { executeUiHighlight };
}
```

4. **Upgrade** `AIAssistantDock.jsx` **(** `frontend-ver2/src/components/AIAssistantDock.jsx` **):**

   Replace the broken static chat call in `src/api.js` with `POST /api/agent/interact`, sending `{ user_message, ui_context: snapshot.uiContext, available_actions: snapshot.availableActions, chat_history }`. ZIP + 1

   When the response arrives:

   a. Render `data.reply` in the message stream. TXT

   b. If `data.campsite_results?.length > 0`, render interactive mini-cards inside the chat dock with rating badges and direct click-to-navigate links.

   c. If `data.trigger_action_id`, invoke `executeAction(data.trigger_action_id, data.action_args)` directly in React state. TXT

   d. If `data.ui_highlight`, invoke `executeUiHighlight(data.ui_highlight)`. TXT

Task F2: Instagram-Style Visitor Stories Suite

Files to Create/Modify:

Create `frontend-ver2/src/components/StoryBar.jsx`

Create `frontend-ver2/src/components/StoryViewerModal.jsx`

Create `frontend-ver2/src/components/StoryUploadModal.jsx`

Modify `frontend-ver2/src/components/CamperStories.jsx`

ZIP + 1

1. `StoryBar.jsx` :

   Horizontal scrollable bar of circular avatars (`64px` diameter) wrapped in an animated emerald-to-cyan gradient ring (`from-[#10b981] to-[#06b6d4]`) for unviewed stories, and a muted slate border (`border-zinc-700`) once viewed (tracked in `localStorage` by `story_slide_id`). MD

   First item is `#btn-upload-story` (`+ Add Moment`) for authenticated users, which opens `StoryUploadModal.jsx` . MD + 1

2. `StoryViewerModal.jsx` :

   Full-screen backdrop (`z-50 bg-black/95 backdrop-blur-xl`) using `framer-motion` . ZIP + 1

   Segmented Progress Header: Renders N top progress bars for the active user's slides. Uses a `requestAnimationFrame` / `5000ms` interval that pauses on pointer-down (hold-to-pause) and auto-advances to the next slide or next user's story group on completion. MD

   Navigation & Controls: Left 30% tap zone goes to previous slide; right 70% tap zone advances; `ArrowLeft` , `ArrowRight` , and `Escape` keys bound. MD

   Footer Overlay: Displays the photo `caption` , Gemini Vision `ai_tags` as translucent pill badges (`#stargazing` , `#lakefront`), and an optimistic heart toggle button wired to `POST /api/photos/{id}/like` . MD

3. `StoryUploadModal.jsx` :

   Drag-and-drop file input with instant local preview.

   Instruments `useAgentAction` with `id: 'submit_campsite_story'` so the user can tell the AI assistant *"Post my story with caption 'Chilly night by the lake'"* and have the modal populate and submit automatically. TXT

Task F3: Rich Property Hosting, Multi-Image Bento & Share Suite

Files to Create/Modify:

Create `frontend-ver2/src/components/SharePropertyModal.jsx`

Modify `frontend-ver2/src/pages/NewCampground.jsx`

Modify `frontend-ver2/src/pages/EditCampground.jsx`

Modify `frontend-ver2/src/pages/CampgroundDetails.jsx`

Modify `frontend-ver2/src/pages/Dashboard.jsx`

ZIP + 1

1. **Rich Hosting Forms (** `NewCampground.jsx` **&** `EditCampground.jsx` **):**

   Multi-Image Uploader: Support primary cover image + up to 6 gallery images with drag-to-reorder and remove buttons. MD

   Interactive Amenities Grid: Toggleable pill cards for `tent_pitches` , `campfire` , `hot_showers` , `potable_water` , `wifi` , `pet_friendly` , and `electricity` , serializing the selected keys into the `amenities` array. MD

   Address & Rules Inputs: Structured inputs for `location_address` and `rules` (check-in window, max guests, tent capacity). MD

   Register `useAgentAction({ id: 'fill_campground_form', ... })` so the AI assistant can pre-fill amenities, title, or rules from natural language prompts. TXT

2. **One-Click Share Suite (** `SharePropertyModal.jsx` **):**

   Triggered via `#btn-share-property` on `CampgroundDetails.jsx` . MD + 1

   Copy Direct Link: Uses `navigator.clipboard.writeText(window.location.href)` with a 2-second `"Copied to clipboard!"` state badge. MD

   Social Intents: Direct share launchers for WhatsApp (`[https://wa.me/?text=](https://wa.me/?text=)...`), X/Twitter (`[https://twitter.com/intent/tweet?url=](https://twitter.com/intent/tweet?url=)...`), Telegram, and Facebook. MD

   Dynamic Canvas QR Code: Renders a clean scannable QR code on an HTML5 `<canvas>` (with a `"Download QR"` button) so campers can scan the property directly from a friend's phone. MD

   Open Graph Card Preview: Live visual preview card displaying the cover photo, title, price, rating, and location badge as it appears in messaging apps. MD

3. **Upgrade** `CampgroundDetails.jsx` **&** `Dashboard.jsx` **:**

   `CampgroundDetails.jsx` **:** Mount `<StoryBar/>` directly above the hero gallery; upgrade the single image into a 5-photo Bento Grid with a

`"View All Photos"` lightbox; display the structured amenity icons and house rules; and render recursive nested replies under reviews. ZIP + 2

`Dashboard.jsx` : Add two distinct tabs—**"My Hosted Properties"** (manage listings, edit details, view story stats) and **"My Uploaded Moments"** (grid of the user's active/archived visitor photos with delete controls). ZIP + 1

Part 4: Frontend-Backend Integration Matrix & Execution Order

Step	Layer	Component / Module	Verificati
1	Backend	`models.py` + `turso_engine.py`	Run `seed` `seed_ve` `campgro` `idx_cam` `ui_navi` populate errors. ZI
2	Backend	`vision_service.py` + `visitor_photos.py`	Test `POST /api/ca` via Swagg URL, Gem and backg embeddir
3	Backend	`vector_store.py` + `vector_rag.py`	Test `POST /a` with nega and rare l verify Flas positives.
4	Frontend	`AgentContext.jsx` + `AIAssistantDock.jsx`	Ask chat " *campsite* navigates and highli `#btn-sh` MD + 1
5	Frontend	`StoryBar.jsx` + `StoryViewerModal.jsx` + `SharePropertyModal.jsx`	Upload a emerald-c 5-second viewer, lik QR code s

If you paste or re-upload the raw code of any `backend/routers/vector_rag.py` or `frontend-ver2/src/components/AIAssis` replacement file immediately.

ZIP TXT

yelpcamp_analysis) property_sharing_and_visitor_stories_p... h To build a production-grade Retriev

Iterate throught this zip , find flaws etc and reprompt it throught the first one , chat bot isnt behaving property , chekc that ifs that static / harcoded . A lot feature are implement and yet to be implemented , implemented features are broken {either does not work pproperty like chat system a lot to imporve as given in upload files} , critque all of previous code , also see To be added feature in both plans / upload context , give detailed task step wiht exceptional specificaitn both for frontend and backend separte yet keeping both compatible

Iterating through every entry in `yelpcamp_analysis.zip` (`source: 4`) alongside your two architecture specifications (`source: 5` , `source: 6`) reveals why the chatbot feels static/hardcoded and why several features fail at runtime.

(Note: Because `yelpcamp_analysis.zip` was uploaded as a raw binary deflate stream, its Central Directory file table is readable while the compressed file payloads are binary-garbled. However, cross-referencing the exact file tree and byte sizes in `source: 4` against your two design specs exposes every structural flaw and missing file.)

Part 1: File-by-File Critique & Why the Chatbot Is Static/Broken

1. Diagnosis: Why the Chatbot (`AIAssistantDock.jsx` + `vector_rag.py`) Behaves Like a Static/Hardcoded Widget

Zero Runtime UI Awareness (`frontend-ver2/src/context/`): Looking at `frontend-ver2/src/context/` in `source: 4` , the **only** context file that exists is `AuthContext.jsx` . Neither `AgentProvider.jsx` (`AgentContext`) nor `useAgentAction.js` exists in the repository. Because of this, `AIAssistantDock.jsx` sends plain text queries to `vector_rag.py` with **zero knowledge** of what page the user is on, which campground is open, what modal is mounted, or what form fields exist.

Hardcoded / Disconnected Action Handling: In `AIAssistantDock.jsx`, starter prompts and responses cannot trigger React state handlers or highlight DOM elements because `driver.js` is not wired into a `useAssistantActions` hook (`frontend-ver2/src/hooks/` does not exist at all in `source: 4`).

The "Always-Return-1-Row" Vector Bug in `vector_store.py`: Your backend relies on `vector_top_k` without a cosine distance cutoff (`WHERE vector_distance_cos(...) < 0.25`) or a Cross-Encoder reranker (`FlashRank`). As a result, when you ask the bot *"How do I delete my story?"* or *"Show me quiet camps with no DJ"*, `vector_top_k` blindly returns the closest mathematical vector (often the exact opposite action or a loud party camp) or falls back to a static canned response.

Static Seed Vectors (`seed_vectors.py`): Embeddings are only generated when you manually run `backend/seed_vectors.py`. When a user creates a new campground (`campgrounds.py`), posts a comment (`comments.py`), or shares a social post (`social.py`), the live endpoints do **not** run a `BackgroundTasks` pipeline to update the `F32_BLOB(768)` column or an FTS5 keyword index. Thus, the chatbot only ever knows about static seeded data.

2. Full Codebase Audit Across `backend/` & `frontend-ver2/`

Existing File in `yelpcamp_analysis.zip`	Architectural Flaw / Broken Behavior	Missing Counterpart
`backend/database.py` & `backend/turso_engine.py`	**Dual-Engine Split Brain:** SQLModel (`database.py`) manages relational tables while `turso_engine.py` runs raw SQLite/libSQL queries. Schema migrations (`amenities`, `gallery_images`, `F32_BLOB`) are not unified.	Unified startup DDL in with `F32_BLOB(768)` `FTS5` virtual tables.
`backend/models.py` & `backend/schemas.py`	`Campground` lacks `gallery_images`, `amenities`, `location_address`, `rules`,	`VisitorPhoto` (with TTL) and `VisitorPho` key models.

Existing File in `yelpcamp_analysis.zip`	Architectural Flaw / Broken Behavior	Missing Counterpart
	`ai_media_caption`, and `community_summary`. `Comment` lacks `parent_comment_id` for nested threads.	
`backend/routers/social.py`	Monolithic placeholder for social features; lacks Gemini Vision auto-tagging, user-grouped story rings, and 24-hour story expiration filtering.	`backend/routers/` & `backend/services/` (completely missing fr
`backend/routers/comments.py`	Flat single-level comments; does not recalculate `avg_rating` on `Campground` or aggregate 5+ nested replies into a `community_summary` vector.	Recursive CTE (`WITH` flattener + debounced embedder.
`frontend-ver2/src/components/CamperStories.jsx`	Hardcoded/static showcase component on the landing page; clicking avatars does not open a segmented 5-second auto-advancing story player.	`StoryBar.jsx`, `StoryViewerModal` `StoryUploadModal`
`frontend-ver2/src/pages/CampgroundDetails.jsx`	Single hero image display, flat comments, no story bar above gallery, and no property sharing modal.	`SharePropertyMod` WhatsApp/X, Canvas G preview).
`frontend-ver2/src/pages/NewCampground.jsx` & `EditCampground.jsx`	Basic single-image URL/upload form; lacks multi-photo drag-and-drop gallery, interactive amenity pills, and AI form-filling hooks.	Multi-image uploader + picker + `useAgentAc`

Part 2: Master Re-Prompt (Copy-Paste into Your Coding IDE / Agent)

Use the following **Unified Master Prompt** to instruct your coding environment (or re-prompt from scratch) so that every broken feature and both architectural plans are implemented with 100% frontend-backend compatibility:

> *MASTER ARCHITECTURAL PROMPT: YelpCamp Full-Stack Refactor & Agentic RAG Upgrade*
>
> **Objective:** *Fix all broken/static features in* `backend/` *and* `frontend-ver2/` *and implement* **Plan 1 (Property Sharing, Multi-Image Hosting & Instagram-Style Visitor Stories)** *and* **Plan 2 (Turso 3-Stage Hybrid Vector+FTS5 RAG & Dynamic React State-Machine UI Assistant)** *with strict API contract compatibility.*
> **Core Rules:**
>
> 1. **No Static/Hardcoded Chatbot Responses:** *Remove all canned responses in* `AIAssistantDock.jsx` *and* `vector_rag.py`*. Every chat message must pass the live React* `AgentContext` *snapshot (* `ui_context` *+* `available_actions` *) to* `POST /api/agent/interact`*.*
> 2. **3-Stage Hybrid Retrieval Only:** *Never use raw* `vector_top_k` *alone. Always combine (a) Turso* `vector_top_k` *with* `vector_distance_cos < 0.35`*, (b) SQLite* `FTS5` *BM25 keyword search for rare place names, and (c)* `FlashRank` *Cross-Encoder reranking (* `score >= 0.25` *) to catch negations.*
> 3. **Non-Blocking Media Ingestion:** *All Gemini 3.5 Flash Vision analysis (* `vision_service.py` *) and Gemma 2 768-d embedding calls (* `multimodal_embedder.py` *) on campground/story creation and comment aggregation must run asynchronously or via FastAPI* `BackgroundTasks`*.*

Part 3: Exceptional Step-by-Step Backend Specifications (`backend/`)

Step B1: Database Models & Unified Turso Vector/FTS5 Schema

Target Files: `backend/models.py`, `backend/schemas.py`, `backend/turso_engine.py`

1. **Update** `backend/models.py`**:**

Python

```python
from datetime import datetime, timezone
from typing import Optional, List
from sqlmodel import SQLModel, Field, Relationship

class Campground(SQLModel, table=True):
    __tablename__ = "campground"
    id: Optional[int] = Field(default=None, primary_key=True)
    title: str = Field(index=True)
    price: float
    description: str
    location: str
    location_address: Optional[str] = Field(default="")
    image: str
    gallery_images: str = Field(default="[]")     # JSON array of image URLs
    amenities: str = Field(default="[]")          # JSON array: ["wifi","camp
    rules: Optional[str] = Field(default="")      # Check-in rules, max guest
    avg_rating: float = Field(default=0.0)
    ai_media_caption: Optional[str] = Field(default="")
    community_summary: Optional[str] = Field(default="")
    author_id: int = Field(foreign_key="user.id")

class VisitorPhoto(SQLModel, table=True):
    __tablename__ = "visitor_photos"
    id: Optional[int] = Field(default=None, primary_key=True)
    campground_id: int = Field(foreign_key="campground.id", index=True)
    user_id: int = Field(foreign_key="user.id", index=True)
    user_username: str
    user_avatar: Optional[str] = Field(default=None)
    media_type: str = Field(default="photo")      # "photo" | "video"
    image_url: str
    caption: Optional[str] = Field(default="")
    ai_tags: str = Field(default="[]")            # JSON list e.g. ["campfire
    ai_media_caption: Optional[str] = Field(default="")
    likes_count: int = Field(default=0)
    created_at: str = Field(default_factory=lambda: datetime.now(timezone.utc
    expires_at: Optional[int] = Field(default=None, index=True) # Unix timest

class VisitorPhotoLike(SQLModel, table=True):
    __tablename__ = "visitor_photo_likes"
    user_id: int = Field(foreign_key="user.id", primary_key=True)
    photo_id: int = Field(foreign_key="visitor_photos.id", primary_key=True)

class Comment(SQLModel, table=True):
```

```
    __tablename__ = "comment"
    id: Optional[int] = Field(default=None, primary_key=True)
    campground_id: int = Field(foreign_key="campground.id", index=True)
    author_id: int = Field(foreign_key="user.id")
    author_username: str
    parent_comment_id: Optional[int] = Field(default=None, foreign_key="comme
    rating: Optional[int] = Field(default=None)   # 1 to 5 on top-level revie
    content: str
    created_at: int = Field(default_factory=lambda: int(datetime.now(timezone
```

2. **Initialize Vector Columns, DiskANN Indexes & FTS5 in** `backend/turso_engine.py` **:**
   Run an idempotent migration routine during FastAPI `lifespan` startup:

   Add `embedding F32_BLOB(768)` to `campground` , `visitor_photos` , and `ui_navigation` .

   Create DiskANN indexes:

   ```
   CREATE INDEX IF NOT EXISTS campground_vec_idx ON campground
   (libsql_vector_idx(embedding));
   ```

   and

   ```
   CREATE INDEX IF NOT EXISTS stories_vec_idx ON visitor_photos
   (libsql_vector_idx(embedding));
   ```

   .

   Create the BM25 keyword table:

   SQL

   ```
   CREATE VIRTUAL TABLE IF NOT EXISTS campground_fts USING fts5(
       title, location, location_address, description, amenities, ai_media_
       content='campground', content_rowid='id'
   );
   ```

Step B2: Gemini 3.5 Flash Vision Service & Async Ingestion

Target Files: Create `backend/services/vision_service.py` ; Update `backend/services/multimodal_embedder.py`

1. `backend/services/vision_service.py` **:**

Create `async def analyze_image_bytes(image_bytes: bytes, mime_type: str = "image/jpeg") -> dict` using `google.genai.Client()` and model `"gemini-3.5-flash"` (with fallback to `"gemini-2.5-flash"` if environment key targets 2.5).

Return structured JSON: `{"caption": str, "ai_tags": list[str], "visual_description": str}`.

2. `backend/services/multimodal_embedder.py`:

Expose `get_gemma_embedding(text: str) -> list[float]` returning a normalized 768-dimension float array.

Expose `async def process_campground_ingestion(campground_id: int)` and `async def process_story_ingestion(photo_id: int)` for `BackgroundTasks`. Each function downloads/reads the image, calls `analyze_image_bytes`, combines text + visual description + amenities, writes the `vector32(?)` blob into Turso, and refreshes `campground_fts`.

Step B3: 3-Stage Hybrid Search & Dynamic UI Agent Endpoint

Target Files: `backend/services/vector_store.py`, `backend/services/rag_service.py`, `backend/routers/vector_rag.py`

1. **3-Stage Hybrid Search (`backend/services/vector_store.py`):**

Python

```python
from flashrank import Ranker, RerankRequest
ranker = Ranker(model_name="ms-marco-MiniLM-L-12-v2")

def hybrid_search_campsites(db, query: str, min_rating: float = 0.0, include_
    q_vec = json.dumps(get_gemma_embedding(query))
    now = int(time.time())

    # Stage 1: Dense Vector Search with Distance Cutoff (< 0.38)
    vec_rows = db.execute("""
```

```
        SELECT c.id, c.title, c.location, c.price, c.avg_rating, c.image,
               c.description, c.amenities, c.ai_media_caption, c.community_su
        FROM vector_top_k('campground_vec_idx', vector32(?), 15) v
        JOIN campground c ON c.rowid = v.id
        WHERE c.avg_rating >= ?
          AND vector_distance_cos(c.embedding, vector32(?)) < 0.38
    """, (q_vec, min_rating, q_vec)).fetchall()

    # Stage 2: SQLite FTS5 BM25 Keyword Search (Catches rare nouns like 'Muns
    fts_query = " OR ".join([w for w in query.replace('"', '').split() if len
    fts_rows = db.execute("""
        SELECT c.id, c.title, c.location, c.price, c.avg_rating, c.image,
               c.description, c.amenities, c.ai_media_caption, c.community_su
        FROM campground_fts f
        JOIN campground c ON c.id = f.rowid
        WHERE campground_fts MATCH ? AND c.avg_rating >= ?
        ORDER BY bm25(campground_fts) LIMIT 15
    """, (fts_query, min_rating)).fetchall() if fts_query else []

    # Deduplicate candidates
    candidates = {}
    for row in list(vec_rows) + list(fts_rows):
        d = dict(row)
        candidates[d["id"]] = {
            "id": d["id"],
            "text": f"{d['title']} in {d['location']}. {d['description']} Ame
            "meta": d
        }

    if not candidates:
        return []

    # Stage 3: FlashRank Cross-Encoder Reranking (Catches negations like 'no
    rerank_req = RerankRequest(query=query, passages=list(candidates.values()
    ranked = [r for r in ranker.rerank(rerank_req) if r["score"] >= 0.25]
    return [r["meta"] for r in ranked[:5]]
```

2. **UI Navigation Lookup with Distance Guardrail (** `backend/services/vector_store.py` **):**

Python

```
def find_ui_navigation_target(db, user_goal: str) -> Optional[dict]:
    q_vec = json.dumps(get_gemma_embedding(user_goal))
```

```
    row = db.execute("""
        SELECT u.feature_name, u.route, u.dom_selector, u.description
        FROM vector_top_k('ui_nav_idx', vector32(?), 1) v
        JOIN ui_navigation u ON u.rowid = v.id
        WHERE vector_distance_cos(u.embedding, vector32(?)) < 0.25
    """, (q_vec, q_vec)).fetchone()
    return dict(row) if row else None
```

3. **Dynamic Agent Endpoint (`POST /api/agent/interact` in `backend/routers/vector_rag.py`):**

   Replace any static/hardcoded response logic in `vector_rag.py`.

   Accept `AgentInteractRequest`:

   JSON

```
{
  "user_message": "string",
  "ui_context": { "current_route": "/campgrounds/12", "active_campground
  "available_actions": [
    { "id": "open_share_modal", "description": "Opens the property shari
  ],
  "chat_history": []
}
```

   Pass `hybrid_search_campsites` and `find_ui_navigation_target` as Python tools to Gemini 3.5 Flash alongside the dynamic `available_actions` injected into the `system_instruction`.

   Return strict JSON:

   JSON

```
{
  "reply": "Conversational response grounded in real DB results and acti
  "campsite_results": [...],
  "story_results": [...],
  "trigger_action_id": "open_share_modal",
  "action_args": {},
  "ui_highlight": {
```

```
    "route": "/campgrounds/12",
    "dom_selector": "#btn-share-property",
    "feature_name": "Share Property",
    "description": "Click here to copy the link, share on WhatsApp, or s
  }
}
```

Step B4: Visitor Photos Router, Rich Campgrounds & Recursive Comments

Target Files: Create `backend/routers/visitor_photos.py` ; Update `backend/routers/campgrounds.py` , `backend/routers/comments.py` , `backend/main.py`

1. `backend/routers/visitor_photos.py` :

   `POST /api/campgrounds/{id}/photos` : Accepts `file: UploadFile` , `caption: Optional[str] = Form("")` , `is_story: bool = Form(True)` . Uploads image via `cloudinary_service.py` , runs `vision_service.analyze_image_bytes` to populate `ai_tags` and fallback `caption` , sets `expires_at = int(time.time()) + 86400` if `is_story` , inserts `VisitorPhoto` , and schedules `BackgroundTasks` vector embedding .

   `GET /api/campgrounds/{id}/stories` : Selects unexpired photos (`expires_at IS NULL OR expires_at > unixepoch()`) for `campground_id` , groups them by `user_id` , and returns each user's avatar ring object with their `photos` array and `liked_by_me` boolean.

   `GET /api/stories/explore` : Returns top active stories across all campgrounds for the Landing/Explore story bar.

   `POST /api/photos/{photo_id}/like` : Idempotent toggle on `VisitorPhotoLike` .

   `DELETE /api/photos/{photo_id}` : Author or campground host deletion.

2. `backend/routers/comments.py` :

   Support `parent_comment_id` in `POST /api/campgrounds/{id}/comments` .

   Return nested tree structure in `GET /api/campgrounds/{id}/comments` .

When a campground accumulates 5 new comments since last summary, run a background task with `WITH RECURSIVE thread AS (...)` in SQLite, summarize the community sentiment via Gemini Flash into `campground.community_summary`, and re-embed `campground.embedding`.

Part 4: Exceptional Step-by-Step Frontend Specifications (`frontend-ver2/`)

Step F1: Build the Dynamic Agent State Machine (`AgentContext` & Hooks)

Target Files:

Create `frontend-ver2/src/context/AgentContext.jsx`

Create `frontend-ver2/src/hooks/useAgentAction.js`

Create `frontend-ver2/src/hooks/useAssistantActions.js`

Update `frontend-ver2/src/App.jsx` & `frontend-ver2/src/components/AIAssistantDock.jsx`

1. `frontend-ver2/src/context/AgentContext.jsx`:

   Implement `AgentProvider` exactly as architected in `source: 6`, tracking `registeredActions` and `uiContext`.

   Listen to `useLocation()` from `react-router-dom` so `uiContext.current_route` updates automatically on every page transition.

   Register global navigation actions out-of-the-box (`navigate_home`, `navigate_explore`, `navigate_new_campground`, `navigate_dashboard`) so the AI can navigate even when specific page modals are unmounted.

2. `frontend-ver2/src/hooks/useAgentAction.js` & `useAssistantActions.js`:

   `useAgentAction(action)` registers a component capability on mount and cleans it up on unmount.

   `useAssistantActions()` wraps `driver.js` (`npm install driver.js`) to navigate to `ui_highlight.route` and pulse-highlight `ui_highlight.dom_selector` after a `300ms` render delay.

3. **Refactor** `frontend-ver2/src/components/AIAssistantDock.jsx`:

   Strip out all hardcoded responses or mock timers.

Connect to `useAgent()` and `useAssistantActions()`.

On message submit, call `POST /api/agent/interact` via `src/api.js` with `getCurrentSnapshot()`.

Render interactive campsite recommendation cards inside the chat thread, automatically trigger `registeredActions[data.trigger_action_id]?.handler(data.action_args)`, and fire `executeUiAction(data.ui_highlight)`.

Step F2: Instagram-Style Visitor Stories Components

Target Files:

Create `frontend-ver2/src/components/StoryBar.jsx`

Create `frontend-ver2/src/components/StoryViewerModal.jsx`

Create `frontend-ver2/src/components/StoryUploadModal.jsx`

Update `frontend-ver2/src/components/CamperStories.jsx`

1. `StoryBar.jsx`:

   Fetches `/api/campgrounds/${campgroundId}/stories` (when on `CampgroundDetails.jsx`) or `/api/stories/explore` (when on `Landing.jsx` / `CampgroundList.jsx`).

   Renders `#btn-upload-story` (`+ Add Your Story` circle) first for logged-in users, followed by visitor avatar rings with gradient border `bg-gradient-to-tr from-[#10b981] to-[#06b6d4]` for unviewed stories and `border-zinc-700` for viewed stories.

2. `StoryViewerModal.jsx`:

   Full-screen overlay with segmented progress bars at the top (`100%` width over `5000ms` per slide using `requestAnimationFrame`).

   Tap left/right halves or press `ArrowLeft` / `ArrowRight` to switch slides or jump to the next user's story ring.

   Bottom bar renders `caption`, Gemini AI scenery badges (`ai_tags` parsed from JSON), and an animated heart button calling `POST /api/photos/{id}/like`.

3. `StoryUploadModal.jsx`:

Uploads image file + caption to `POST /api/campgrounds/{id}/photos`.

Registers `useAgentAction({ id: 'submit_campsite_story', ... })` and updates `uiContext('story_modal_draft_caption', text)` on input change.

Step F3: Rich Property Creation, Bento Gallery & One-Click Share Suite

Target Files:

Create `frontend-ver2/src/components/SharePropertyModal.jsx`

Update `frontend-ver2/src/pages/NewCampground.jsx`

Update `frontend-ver2/src/pages/EditCampground.jsx`

Update `frontend-ver2/src/pages/CampgroundDetails.jsx`

Update `frontend-ver2/src/pages/Dashboard.jsx`

1. `NewCampground.jsx` & `EditCampground.jsx`:

   Multi-image drag-and-drop uploader (`gallery_images`).

   Interactive 7-amenity selector grid (`Tent Pitches` , `Campfire Allowed` , `Hot Showers` , `Potable Water` , `High-Speed WiFi` , `Pet-Friendly` , `Electricity`).

   Address input (`location_address`) and property rules (`rules`).

   Register `useAgentAction({ id: 'populate_campground_form', ... })` so users can ask the AI assistant to draft their listing details.

2. `SharePropertyModal.jsx`:

   Triggered by `#btn-share-property` in `CampgroundDetails.jsx`.

   One-Click Copy Link with `"Copied to clipboard!"` toast state.

   Social Share Buttons for WhatsApp, X/Twitter, Facebook, and Telegram.

   Dynamic Canvas QR Code generator for mobile trip scanning.

   Open Graph Preview Card showing how the campsite link unfurls in chat apps.

3. `CampgroundDetails.jsx` & `Dashboard.jsx`:

In `CampgroundDetails.jsx` : Mount `<StoryBar campgroundId="{id}"/>` above the multi-photo gallery, render amenity badges, mount `<SharePropertyModal/>` , and render nested recursive comments with a `"Reply"` action per comment.

In `Dashboard.jsx` : Add **"My Properties"** (host management view) and **"My Uploaded Moments"** (visitor stories/photos grid with delete button) tabs.

Part 5: Frontend-Backend Compatibility Contract Matrix

Ensure every frontend API call in `frontend-ver2/src/api.js` matches these exact FastAPI signatures and DOM selectors:

ature	Frontend Component & DOM ID	API Endpoint (`src/api.js`)	Req
gentic Chat & UI ntrol	`AIAssistantDock.jsx` (`#ai-assistant-dock`)	`POST /api/agent/interact`	{ ui ava cha
load Visitor Story	`StoryBar.jsx` (`#btn-upload-story`)	`POST /api/campgrounds/{id}/photos`	For cap
mpground Stories ng	`StoryBar.jsx` & `StoryViewerModal.jsx`	`GET /api/campgrounds/{id}/stories`	Que in
plore Trending Stories	`Landing.jsx` / `CampgroundList.jsx`	`GET /api/stories/explore`	Non
ggle Story Like	`StoryViewerModal.jsx` (	`POST`	Non